

PA1 - 3D Shapes in WebGL

Student name / ID: Islamiya Okassova / 242322

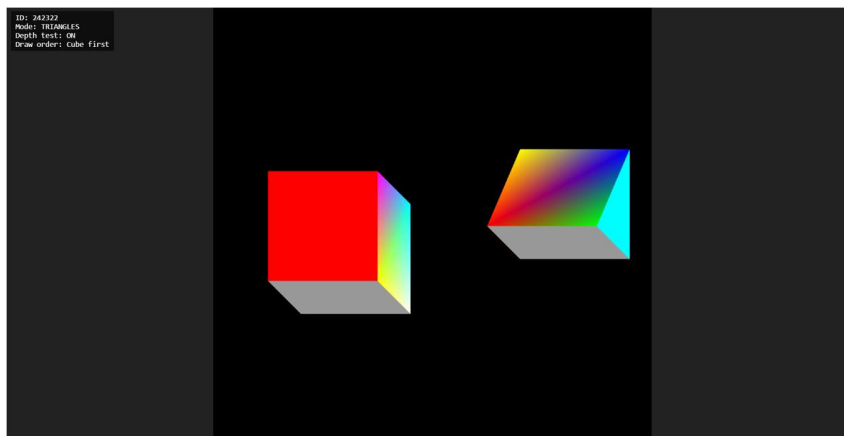
Variant

Solid: wedge (ramp). Offset (o_x, o_y) = (+0.15, -0.15); visible side faces with depth ON: bottom and right. Array order: cube (36 vertices) = front, back, left, right, top, bottom; wedge (24) = bottom, back, slope, left end, right end.

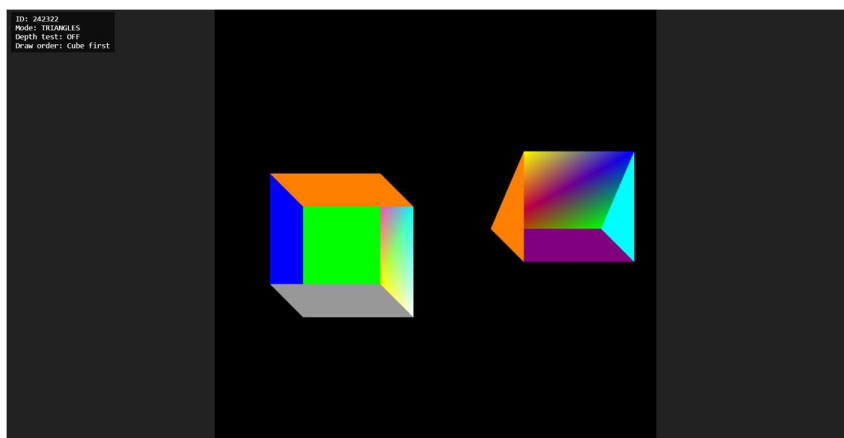
E1 - Depth test vs draw order

Prediction: With depth OFF the last drawn triangle wins at each pixel, so the front face (listed first) should be covered. S should change nothing while the solids share no pixels; if they overlap, the solid drawn last should be on top.

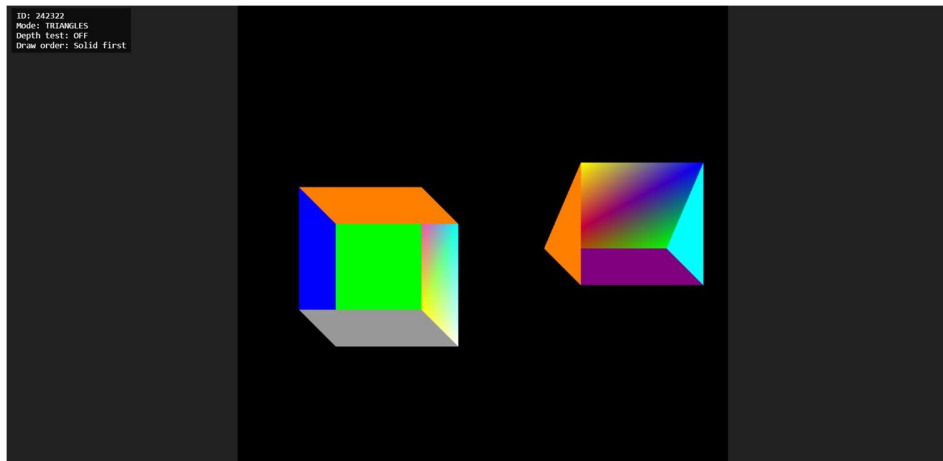
Result: Screenshot 1 (depth ON): the cube shows front (red), right (gradient) and bottom (gray), as the variant says. Screenshot 2 (depth OFF): the cube shows back (green), left (blue), top (orange), right (gradient) and bottom (gray); the front face is not visible. Pressing S at the original positions changed only the label (screenshot 2b). With depth ON, S also changed only the label (screenshot 2c). With overlap and depth OFF, the wedge is on top when drawn last (screenshot 3) and the cube is on top after S (screenshot 4).



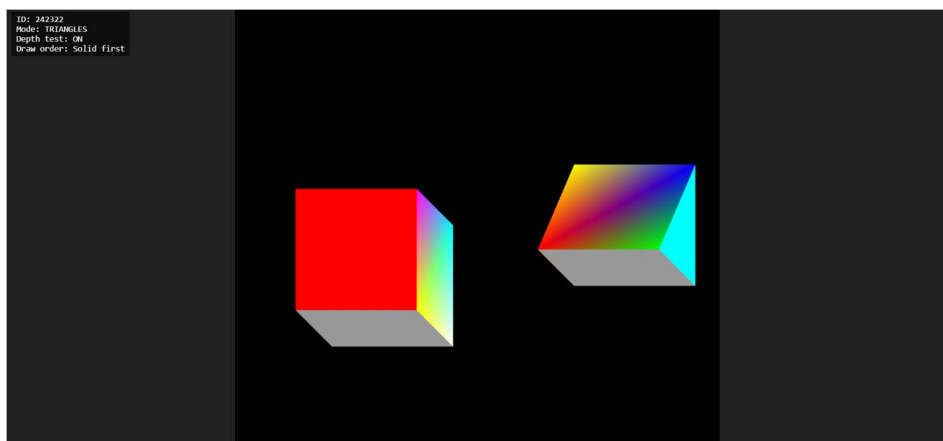
Screenshot 1: depth ON



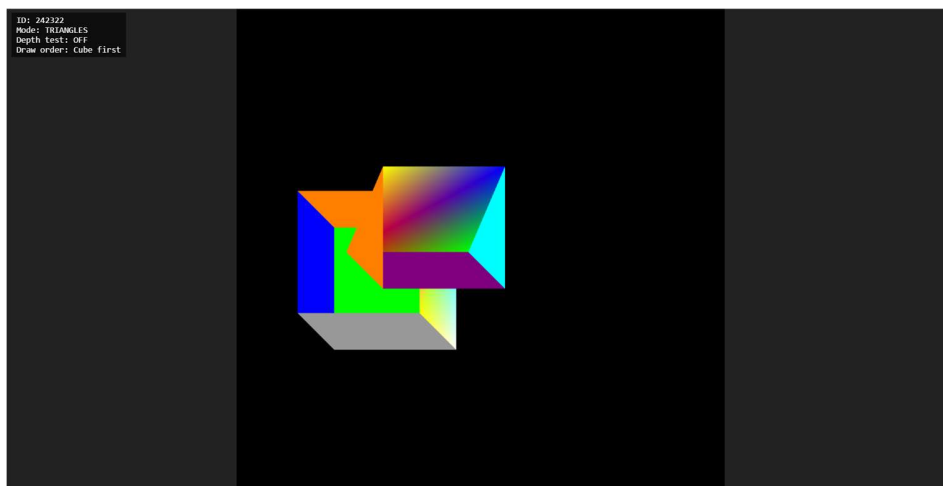
Screenshot 2: depth OFF



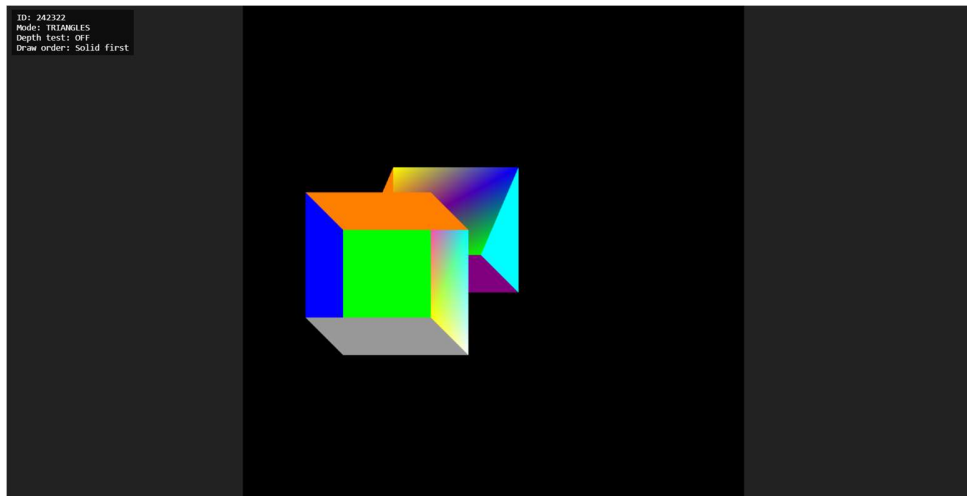
Screenshot 2b: depth OFF, solid first



Screenshot 2c: depth ON, solid first.



Screenshot 3: overlap, cube first



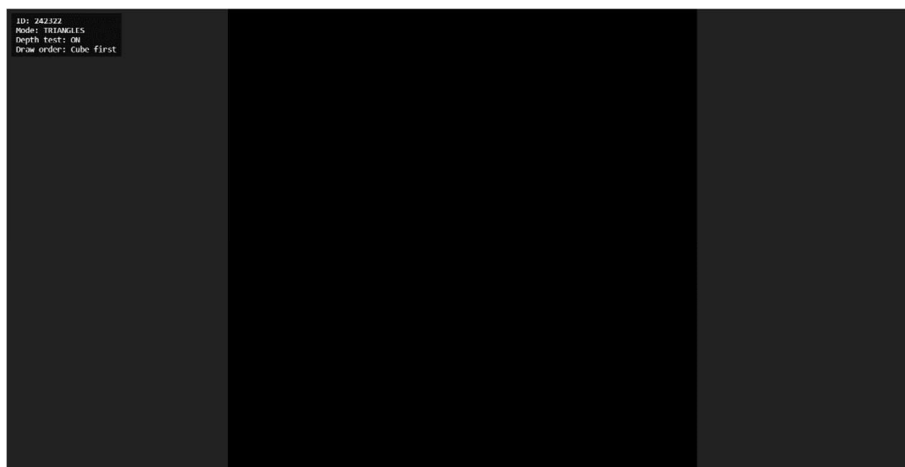
Screenshot 4: overlap, solid first

Explanation: Without the depth test each triangle overwrites the pixels below it, so array order decides. The back, left and top faces come after the front face and together cover it. The cube ($x < 0$) and the wedge ($x > 0$) share no pixels, so call order cannot change any pixel. When they overlap, the last call wins, so S swaps which solid is on top. With depth ON, depth values decide instead.

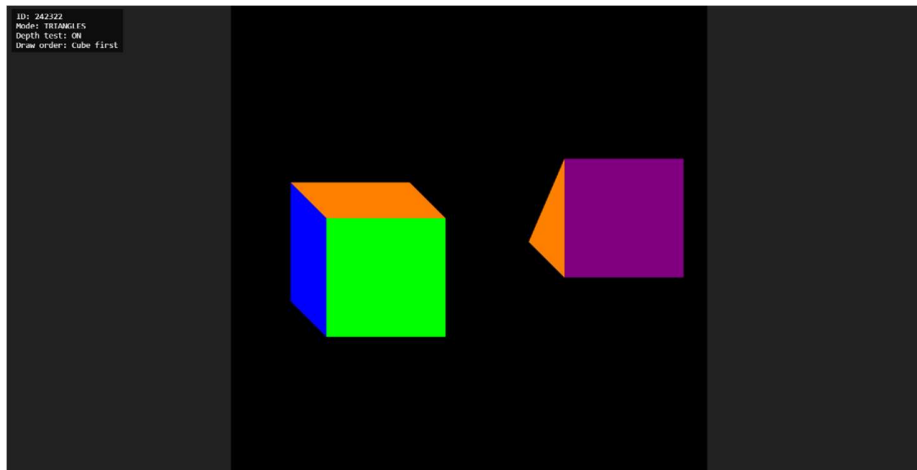
E2 - Why negative z is in front

Prediction: $\text{Depth} = (z + 1) / 2$ and smaller is nearer. The buffer clears to 1.0 and LEQUAL keeps the smaller depth, so $z = -0.3$ wins. (a) GREATER alone: nothing passes. (b) GREATER with `clearDepth(0.0)`: the farthest surface wins.

Result: (a) Screenshot 5: black canvas, only the status label. (b) Screenshot 6: the cube shows back (green), left (blue) and top (orange); the wedge shows back (purple) and left (orange). These are the opposite faces from screenshot 1.



Screenshot 5: GREATER alone



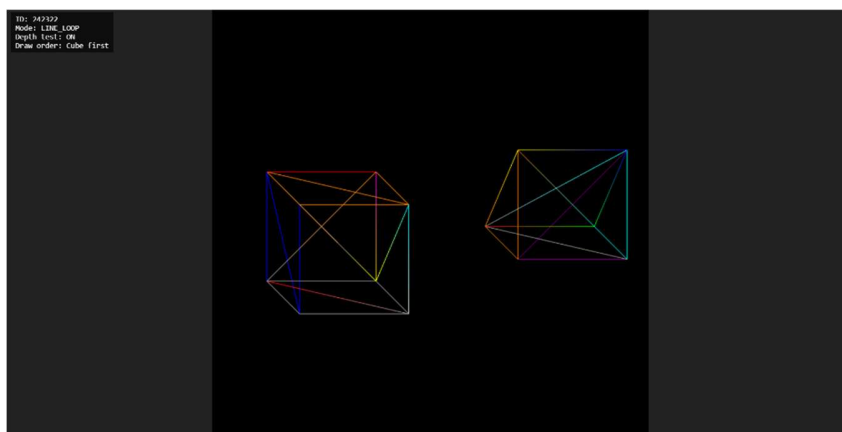
Screenshot 6: GREATER with clearDepth(0.0)

Explanation: Our depth values lie between 0.25 and 0.75. In this WebGL setup, after the vertex positions are used directly as clip-space coordinates, the depth value is mapped from $z = -1$ to the near value 0 and $z = +1$ to the far value 1. Therefore, the smaller z value produces the smaller depth value and is treated as closer by `gl.LEQUAL`. (a) Every pixel holds 1.0 and no fragment exceeds 0.75, so GREATER rejects all. (b) After clearing to 0.0 the first fragment passes and later ones pass only if farther, so the farthest faces remain and the solids are seen from the other side.

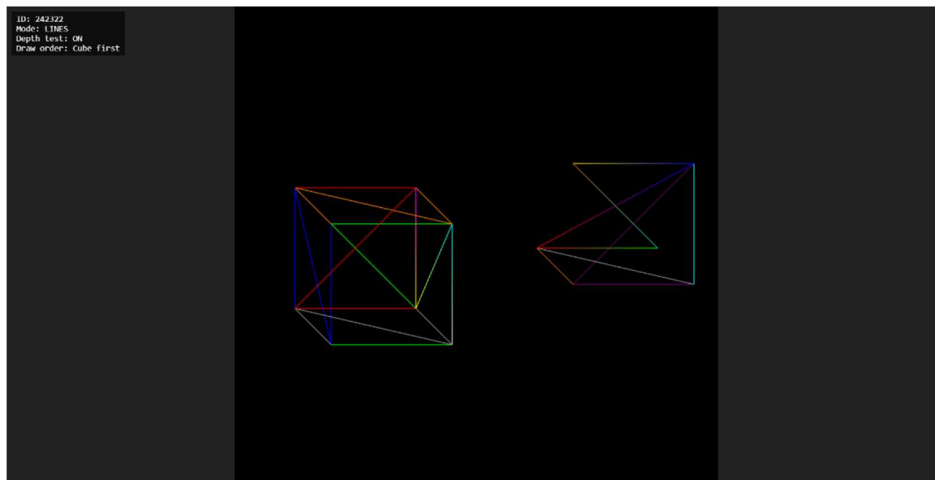
E3 - Drawing modes on triangle data

Prediction: The array holds independent triangles, so `LINE_LOOP` should add extra lines, `LINES` should show only some edges, and `TRIANGLE_STRIP` should build wrong triangles.

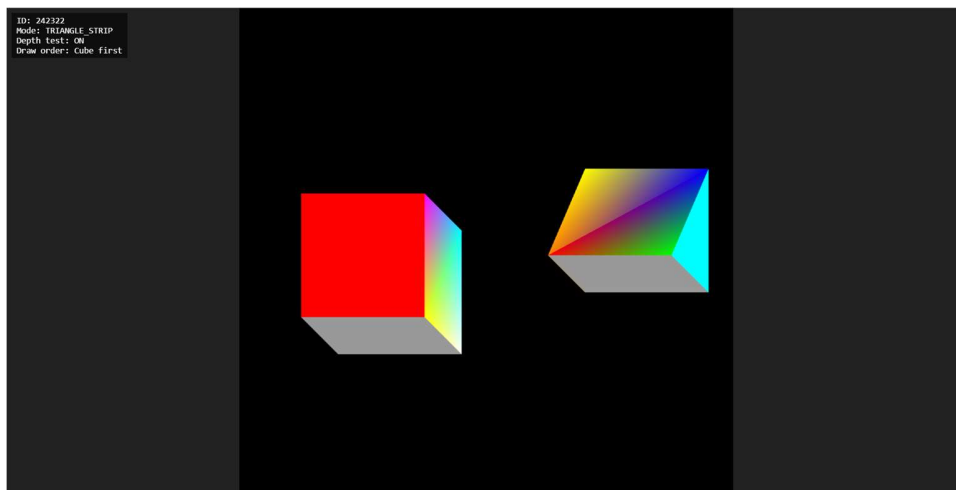
Result: Screenshot 7 (`LINE_LOOP`): wireframes with many extra diagonals, some crossing between faces. Screenshot 8 (`LINES`): fewer lines, incomplete outline. Screenshot 9 (`TRIANGLE_STRIP`, depth ON): almost the same as screenshot 1; I could not see wrong triangles. With depth OFF, screenshot 9b makes the incorrect strip triangles much more visible. The scene contains additional triangular regions because `TRIANGLE_STRIP` connects consecutive vertices across the independently stored triangles and faces.



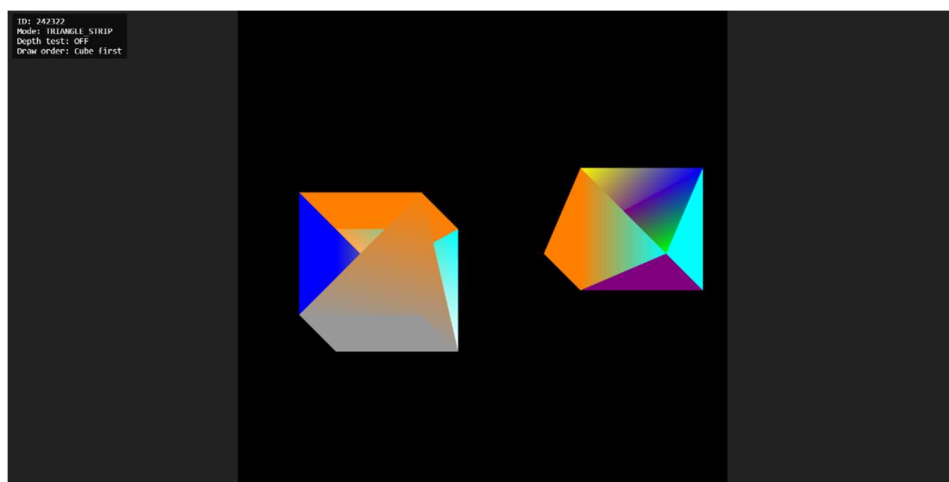
Screenshot 7: LINE_LOOP



Screenshot 8: LINES.



Screenshot 9: Triangle_Strip, depth ON



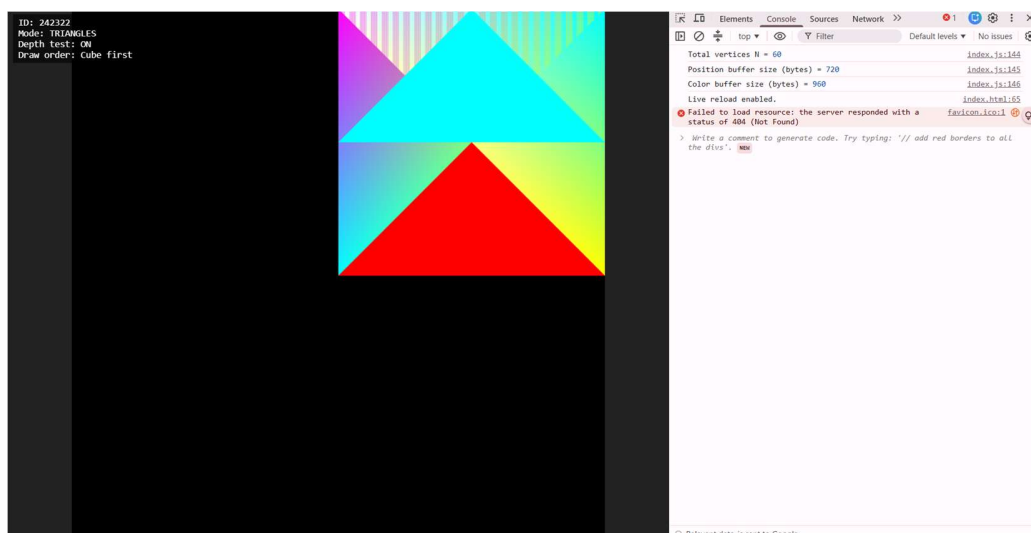
Screenshot 9b: Triangle_Strip, depth OFF

Explanation: Each solid is one `drawArrays` call. `LINE_LOOP` joins consecutive vertices and closes to the first, so besides the edges and one diagonal per face it joins the last vertex of a face to the first of the next, giving lines across the solid. `LINES` joins pairs 0-1, 2-3, 4-5, so a face gets two edges and a diagonal; the other two edges would need pairs 1-2 and 3-4, which are never made. In `TRIANGLE_STRIP` triangle `i` uses vertices `i`, `i+1`, `i+2`. A face is A, B, C, A, C, D, so the original triangles are strip triangles 0 and 3 and are still drawn; the other generated triangles are either duplicates, degenerate triangles with zero area, or triangles that connect vertices from different original faces. Those are wrong, but with depth ON they are almost hidden.

E4 - The missing bind

Prediction: The pointer takes the buffer bound at the call. With `colorBuffer` still bound, `aPosition` should read color data as positions, giving wrong geometry and no error.

Result: In a copy I removed `gl.bindBuffer(gl.ARRAY_BUFFER, buffers.positionBuffer)` before the `aPosition` call. The solids are not recognizable; large bright triangles fill only the upper right quarter. No WebGL error was reported; the distorted geometry was the visible result of the incorrect attribute-buffer binding.



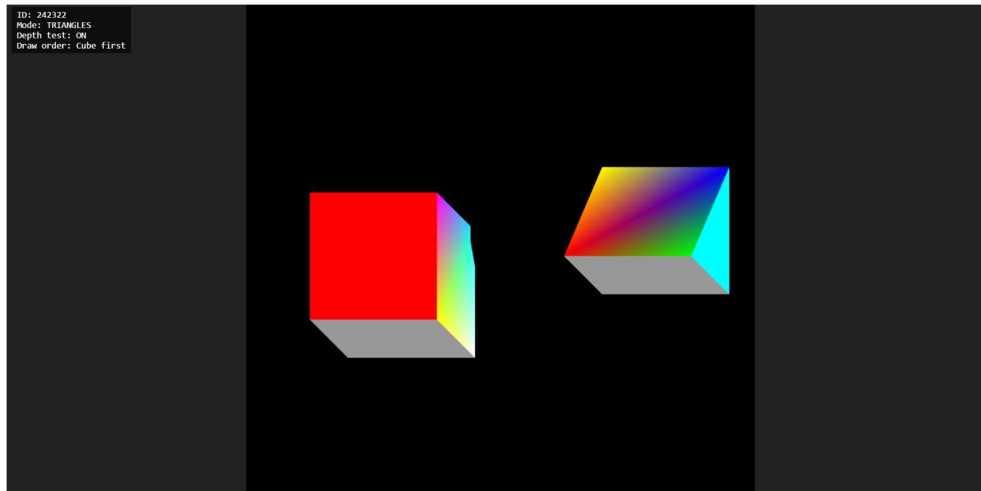
Screenshot 10: colorBuffer bound at the aPosition call

Explanation: The pointer stores the buffer bound when the call ran, here `colorBuffer`, so `aPosition` reads color numbers as x, y, z in groups of three. Color values lie between 0 and 1, so `x, y >= 0` and everything falls in the upper right quarter. No error appears because the call is valid for any bound buffer, and the color buffer (240 floats) holds more than the 180 floats read (60×3).

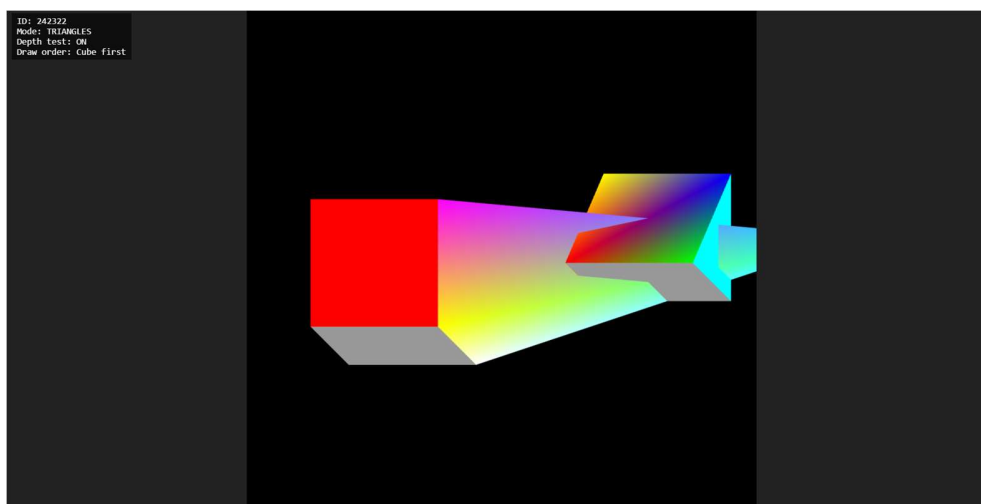
E5 - Clipping

Prediction: The clip-space limits are -1 to +1 on each axis. Moving the vertex to `z = 1.2` should clip the parts of the triangles that extend beyond the `z = 1` boundary. Moving it to `x = 1.4` should push the affected triangles beyond the right side of clip space, so the visible geometry should be cut at the right edge.

Result: I moved the back-top-right vertex of the cube. Screenshot 11: the cube looks almost unchanged; the right edge of the gradient face has a small step near its top. Screenshot 12: triangles using this vertex stretch far right, are cut at the canvas edge and overlap the wedge.



Screenshot 11: one vertex at $z = 1.2$



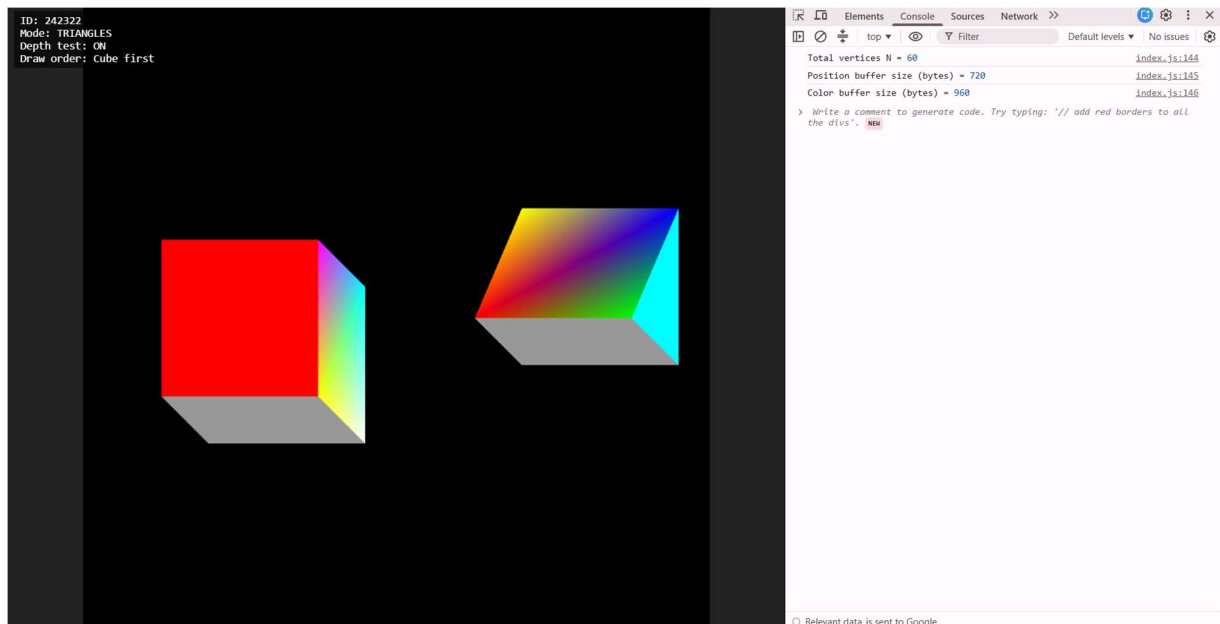
Screenshot 12: one vertex at $x = 1.4$.

Explanation: Without a projection matrix, changing z does not change the horizontal or vertical screen position of the vertex. At $z = 1.2$, the GPU clips the affected triangles where they cross the $z = 1$ boundary, which explains why the visible change is small. The x coordinate directly controls the horizontal clip-space position, so $x = 1.4$ places the vertex outside the right clip boundary. The affected triangles stretch toward that vertex and are clipped where they cross $x = 1$.

E6 - Buffer sizes

Prediction: $N = 36 + 24 = 60$. Position buffer: $60 \times 3 \times 4 = 720$ bytes. Color buffer: $60 \times 4 \times 4 = 960$ bytes.

Result: The console shows $N = 60$ and sizes 720 and 960 bytes, read with `gl.getBufferParameter` (screenshot 13).



Screenshot 13: buffer sizes

Explanation: A 32 bit float takes 4 bytes; positions have 3 floats and colors 4 floats per vertex. `BUFFER_SIZE` reports the currently bound buffer, so each buffer is bound before it is queried.

Development log

1. Bug: README.md was missing in the GitHub repository after the first push.
2. Found by: opening the repository page and seeing only index.html and index.js files.
3. Fix: I added README.md with the run instructions and key map in the second commit.
4. Commit hash: 7024a37